Module Activity: Integrating an External API and MongoDB Database in React Using Express (TMDB API)

Objective: To implement an Express backend that connects to the TMDB (The Movie Database) REST API, integrates MongoDB for data storage, and displays movie data dynamically in a React frontend application using Axios.

1. Introduction: API and Database Integration in Web Apps

In Modern web applications often combine external APIs (for live data) and databases (for user data persistence).

In this activity, you will:

1. Retrieve movie data from TMDB API using Express.
2. Display it in a React frontend.
3. Allow users to save favorite movies to a MongoDB collection.

This integration demonstrates full-stack communication — from React (frontend) → Express (backend) → TMDB API and MongoDB (data storage).

Setup Requirements (Before You Begin)
Before starting your project, make sure you have the following setup completed:

A. TMDB API Key
The TMDB (The Movie Database) API requires a personal developer key for access.

Steps to obtain your key:

1. Visit https://www.themoviedb.org
2. Create a free TMDB account.
3. Go to your Profile → Settings → API.
4. Request an API key under “Developer / Educational Use.”
5. Copy your API key.
6. You will place this in your .env file later as:
    a. TMDB_API_KEY=your_tmdb_api_key_here

B. MongoDB Database

MongoDB Atlas (Online Setup)

1. Go to https://www.mongodb.com/cloud/atlas
2. Create a free account and new cluster.
3. Get your connection string (from “Connect → Drivers”).
4. Replace <username> and <password> with your own credentials, and paste it in your .env file:

5. MONGO_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/tmdb_db

2. Activity Implementation Steps

Step 2.1: Project Setup and Structure

- Create a project folder with two main parts:Install the necessary routing library:

```
project/
├── backend/   → Express server (handles TMDB API calls)
└── frontend/  → React app (displays movie data)
```

Step 2.2: Create the Home Component (Home.jsx)

1. Create the backend folder:
    a. mkdir backend
    b. cd backend
    c. npm init -y
    d. npm install express axios cors dotenv mongoose
2. Create a .env file inside the backend folder:
    a. TMDB_API_KEY=your_tmdb_api_key_here
    b. MONGO_URI=your_mongo_connection_here
    c. PORT=5000
3. Create server.js

```
console.log("🟢 Running server.js from:", process.cwd());


import express from "express";
import axios from "axios";
import cors from "cors";
import dotenv from "dotenv";
import mongoose from "mongoose";

dotenv.config();
const app = express();
app.use(cors());
app.use(express.json());

const TMDB_BASE_URL = "https://api.themoviedb.org/3";

// MongoDB connection
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("✅ Connected to MongoDB"))
  .catch((err) => console.error("❌ MongoDB connection failed:", err.message));
```

```
// MongoDB Schema and Model
const favoriteSchema = new mongoose.Schema({
  title: String,
  poster_path: String,
  release_date: String,
});
const Favorite = mongoose.model("Favorite", favoriteSchema);

// Fetch popular movies from TMDB
app.get("/api/movies/popular", async (req, res) => {
  try {
    const response = await axios.get(`${TMDB_BASE_URL}/movie/popular`, {
      params: { api_key: process.env.TMDB_API_KEY },
    });
    res.json(response.data.results);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✅ Test route (add this too for confirmation)
app.get("/test", (req, res) => {
  res.send("✅ Test route is working");
});

// Save favorite movie to MongoDB
app.post("/api/favorites", async (req, res) => {
  try {
    const movie = new Favorite(req.body);
    await movie.save();
    res.json({ message: "Movie saved to favorites!" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Get all favorite movies
app.get("/api/favorites", async (req, res) => {
  try {
    const favorites = await Favorite.find();
    res.json(favorites);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
// ✅ Keep this at the very bottom
app.listen(process.env.PORT, () =>
  console.log(`✅ Server running on http://localhost:${process.env.PORT}`)
);
```

Step 2.3: Run the Backend

1. Start MongoDB (local or Atlas).
2. Run express backend
    a. node server.js
3. Open these URLs to verify:
    a. http://localhost:5000/api/movies/popular
    b. http://localhost:5000/api/favorites

Step 4: Frontend Setup (React + Vite)

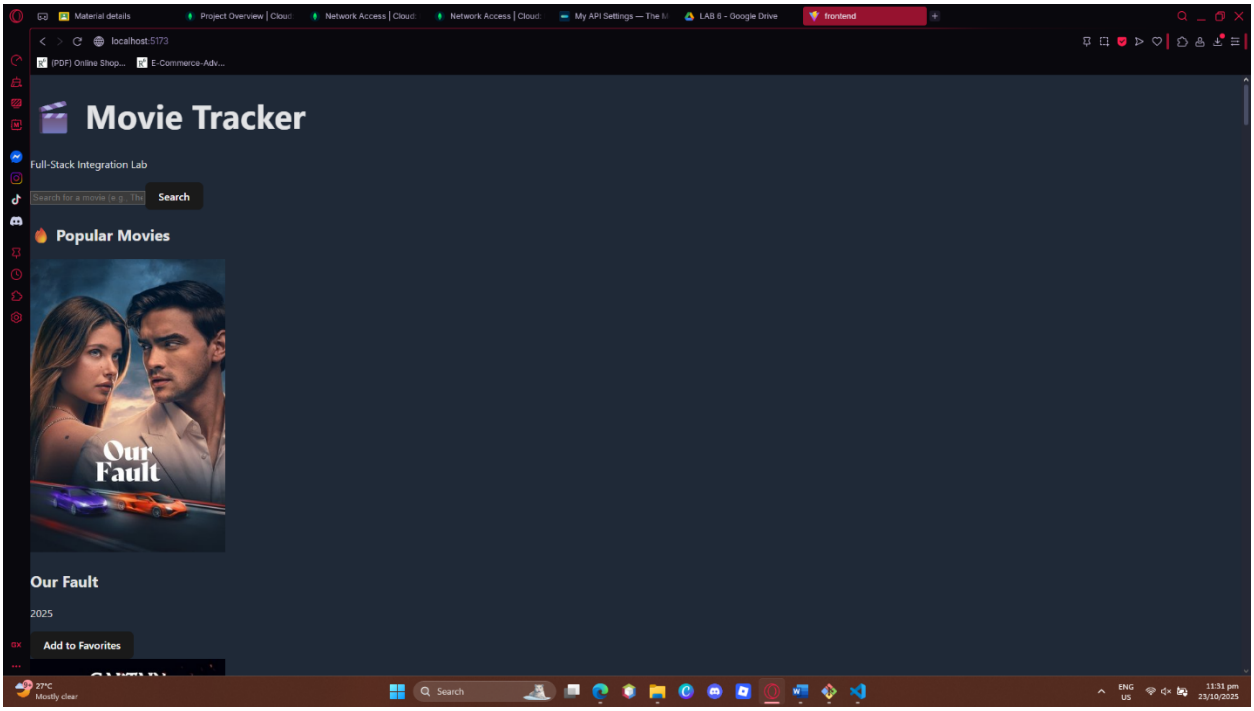
1. Create the React App
    a. npm create vite@latest frontend -- --template react
    b. cd frontend
    c. npm install axios
2. Edit src/App.jsx

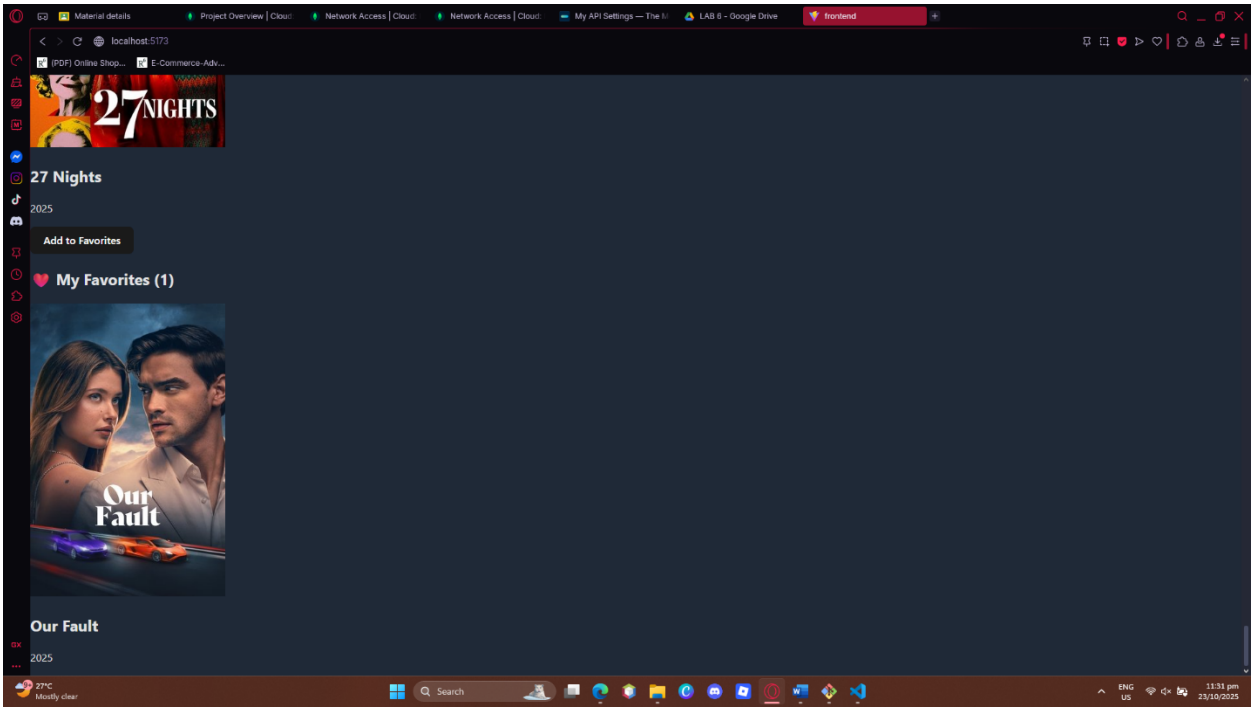
```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [movies, setMovies] = useState([]);
  const [favorites, setFavorites] = useState([]);

  useEffect(() => {
    axios.get("http://localhost:5000/api/movies/popular")
      .then(res => setMovies(res.data))
      .catch(err => console.error(err));

    axios.get("http://localhost:5000/api/favorites")
      .then(res => setFavorites(res.data))
      .catch(err => console.error(err));
  }, []);

  const addFavorite = (movie) => {
    axios.post("http://localhost:5000/api/favorites", {
      title: movie.title,
      poster_path: movie.poster_path,
      release_date: movie.release_date,
    }).then(() => alert("✅ Added to favorites!"));
  };

  return (
    <div className="p-6">
```

```
      <h1 className="text-2xl font-bold mb-4">🎬 Popular Movies</h1>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
       {movies.map(movie => (
        <div key={movie.id} className="bg-gray-100 p-3 rounded shadow-md">
         <img
          src={`https://image.tmdb.org/t/p/w200${movie.poster_path}`}
          alt={movie.title}
          className="rounded mb-2"
         />
         <h2 className="font-semibold text-center">{movie.title}</h2>
         <button
          onClick={() => addFavorite(movie)}
          className="mt-2 bg-blue-500 text-white py-1 px-3 rounded text-sm"
         >
          Add to Favorites
         </button>
        </div>
       ))}
      </div>

      <h2 className="text-xl font-bold mt-8">💔 My Favorites</h2>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mt-2">
       {favorites.map((fav, index) => (
        <div key={index} className="bg-white p-2 rounded shadow-sm">
         <img
          src={`https://image.tmdb.org/t/p/w200${fav.poster_path}`}
          alt={fav.title}
          className="rounded"
         />
         <p className="text-center font-medium">{fav.title}</p>
        </div>
       ))}
      </div>
     </div>
    );
   }

   export default App;
```

3. Run the front end App:
    a. npm run dev
4. Test the full system:
    a. Browse movies from TMDB
    b. Click “Add to favorites”
    c. Check that favorites appear from MongoDB
5. Add search : *(/api/movies/search?query=batman)*
6. Add genre filtering and pagination
7. Make the Popular Movies and My Favorites a Component to be called in App.jsx

/* INSERT THE SCREENSHOT OF YOUR WEBSITE HERE */





Name: _____ **Course, Yr. & Sec:** _____
Student Number: _____ **Date:** _____

DCIT 26 – Application Development and Emerging Technologies
Laboratory Exercise: Integrating External API in React using Express (TMDB API)

Laboratory Grading Rubric			
Criteria	**Definition**	**Points Possible**	**Earned Points**
Task Execution	Successfully created and configured both backend (Express) and frontend (React) components.	20	
Implementation Efficiency & Correctness	Application runs without errors and displays real movie data from TMDB API.	30	
Conceptual Understanding	Clear understanding of API communication flow and data handling.	30	
Code Quality	Code is well-indented, readable, and logically organized.	20	
Total		**100**	